

# Examen Aprendizaje Automático y Minería de Datos

Grado en Desarrollo de videojuegos, enero 2024

## Instrucciones generales

El examen consta de dos partes, una teórica que vale 2 puntos y una práctica que vale 8 puntos.

La parte teórica se entregará en el campus con el nombre del alumno sin espacios y en pdf (Podéis usar word para exportar a PDF Archivo>exportar>crear pdf). No se corregirá si está en otro formato diferente y se calificará como no presentada.

La parte práctica se debe entregar en un Jupiter Notebook dentro de un zip con todos los recursos que necesite para ejecutar. Todo, tanto la implementación como la posible explicación debe estar en el notebook. Para ello podéis hacer uso de las celdas de markdown en caso de necesitar escribir alguna explicación o contestar alguna pregunta. El fichero de Jupiter notebook debe ser ejecutable por si mismo sin necesidad de intervención del profesor, es decir, aseguráros que lo que entreguéis ejecuta y tiene todos los recursos necesarios para que ejecute. La única excepción es el posible uso de librerías, aunque inicialmente no deberíais necesitar ninguna más allá de las vistas en clase. Si la entrega no es correcta y no ejecuta (por ejemplo faltan los datos, las rutas son absolutas y no encuentra los datos del dataset, etc) se penalizará la nota de la parte práctica del examen con hasta -1 puntos.

En el zip también debéis incluir vuestra librería con la implementación del perceptrón multicapa generalizado para múltiples capas y neuronas por capa. Esta librería se debe basar en el código entregado en la práctica 5 y 6 aunque puede tener modificaciones.

Se permite para la realización de la parte práctica y teórica el uso de los recursos del campus, apuntes y cualquier material de apoyo que traigais. No está permitido el uso de ChatGPT o herramientas de IA similares.

## Parte teórica

Tiempo disponible 30 minutos.

Contestad en un documento de texto y exportarlo a PDF. Podéis entregadlo en el campus virtual con vuestro nombre en el nombre del fichero y dentro del mismo como primera línea del archivo. Poned el número de la pregunta que estáis contestando y a continuación la respuesta.

### Pregunta 1 (0.5 puntos)

Si disponemos de los modelos siguientes:

```

self.model01 = torch.nn.Sequential(
    torch.nn.Linear(32 * 32, 128),
    torch.nn.ReLU(),
    torch.nn.Linear(128, 64),
    torch.nn.ReLU(),
    torch.nn.Linear(64, 36),
    torch.nn.ReLU(),
    torch.nn.Linear(36, 18),
    torch.nn.ReLU(),
    torch.nn.Linear(18, 9)
)

```

```

self.model02 = torch.nn.Sequential(
    torch.nn.Linear(9, 18),
    torch.nn.ReLU(),
    torch.nn.Linear(18, 36),
    torch.nn.ReLU(),
    torch.nn.Linear(36, 64),
    torch.nn.ReLU(),
    torch.nn.Linear(64, 128),
    torch.nn.ReLU(),
    torch.nn.Linear(128, 32 * 32),
    torch.nn.Sigmoid()
)

```

Dime de qué tipo de red se trata y explícame para qué podría servir.

### **Pregunta 2 (0.5 puntos)**

Tenemos un agente que se mueve por un espacio de juego discretizado de tamaño 10000x10000 casillas. Este agente parte desde una casilla de salida hasta una casilla donde está situado un cofre que contiene una espada legendaria que necesita para vencer al jefe final. El agente no conoce la situación exacta del cofre. ¿Cómo modelarías el comportamiento del agente usando aprendizaje automático? Explica las alternativas que dispones, posibles problemas y como los solucionarías. Nota: el jugador o diseñador del comportamiento no puede controlar al agente en ningún momento.

### **Pregunta 3 (0.5 puntos)**

Tenemos un modelo de red neuronal que nos da un accuracy de entrenamiento del 60% y un accuracy de validación del 50%. Sabiendo que nuestro baseline es de una precisión de clasificación del 80%. ¿Qué estrategia seguirías para intentar mejorar los resultados?

### **Pregunta 4 (0.5 puntos)**

Disponemos de 200 datos de entrenamiento de una partida guardada por un jugador y con estos datos, queremos construir un modelo de machine learning

que intente imitar al jugador para desarrollar una IA. Los datos que se han guardado son: La posición del jugador (en que casilla está), la acción que ha realizado (moverse a la izquierda, a la derecha, arriba, abajo, disparar, coger item), el contenido de las 8 casillas contiguas al jugador (valores que puede tomar las casillas: vacía, suelo, enemigo, mejora, vida, pared).

Si queremos usar un perceptrón multicapa. ¿Qué número de neuronas de entrada y de salida tendrá el perceptrón?

Si hubiera enemigos que tuvieran un rango de ataque de 2, pero sólo pueden atacar si está exactamente a distancia 2 ¿Qué modificaciones habría que hacer para que el agente pudiera jugar al juego correctamente? Justifica las respuestas. Nota: hay que intentar que la red sea lo más pequeña posible.

## Parte práctica

Tiempo máximo para la realización de la parte práctica 2h y 30 minutos.

Disponemos de un dataset sobre drogas con los siguientes campos:

- Age: edad de los participantes
- Sex: sexo de los participantes
- Blood Pressure Levels (BP): presión sanguínea
- Cholesterol Levels: niveles de colesterol
- Na to Potassium Ration: métrica médica sobre el potasio en sangre.
- drug: Tipo de droga consumida

Queremos construir un modelo que prediga el tipo de droga en función de los datos proporcionados.

**Ejercicio 1 (1 punto):** Limpia el dataset y realiza las transformaciones necesarias para que los datos puedan ser aplicables a cualquier modelo de machine learning. Justifica en una celda de markdown las decisiones que has tomado.

**Ejercicio 2 (1 punto):** Representa gráficamente los datos una vez hayan sido limpiados.

**Ejercicio 3 (2 puntos):** Usa vuestro Perceptrón Multicapa de la práctica 5 y 6 para construir el modelo de predicción. Calcula accuracy y la matriz de confusión. El resultado mínimo que debéis conseguir de precisión es de un 80%. El modelo puede tener cualquier capa, pero **debe existir una prueba realizada con más de una capa oculta**, aunque no sea el modelo definitivo que establezcáis. **Dejad bien claro cual es el modelo con el que finalmente os quedais..** En la partición de datos usad `random_state=13`.

**Ejercicio 4 (1 puntos):** Usa `MLPClassifier` de `SKLearn` para entrenar un modelo. Dicho modelo debe conseguir al menos un 90% de accuracy (`random state = 13`)

**Ejercicio 5 (1 puntos):** Usa el algoritmo KNN para construir un segundo modelo. Calcula accuracy y la matriz de confusión. El resultado mínimo que debéis conseguir de precisión es de un 87.5% pero es posible alcanzar al menos un 90% o incluso más. Cuanto más os acerquéis más puntuación tendrá esta pregunta.

**Ejercicio 6 (1 punto):** Compara escribiendo en una celda en markdown los modelos obtenidos por KNN y por MLPClassifier y elige justificadamente cuál crees que es el que mejor se adapta al problema que queremos modelar y por qué.

**Ejercicio 7 (1 puntos):** Transforma las clases drugA, drugB y drugC en la clase softDrug y las clases drugX y drugY en hardDrug y crea un modelo de clasificación binaria con tu perceptrón multicapa. El performance debe ser similar o mejor al obtenido en el ejercicio 3.

#### **Forma de entrega:**

En el campus virtual con un archivo zip donde esté todo el contenido necesario para ejecutar el jupyter notebook. El zip debe tener vuestro nombre y en la primera línea del notebook también debe aparecer vuestro nombre.

#### **Anexo: API con algunas funciones de Python útiles.**

En Pandas: `isna()` : selecciona campos nulos, combinada con `.sum()` podeis comprobar el número de campos nulos de un dataframe.

`.drop("columna",axis=1)` elimina una columna (igual pero con un índice y `axis 0` elimina una fila)

`dropna(how='any')`: borra campos nulos.

`to_numpy()` convierte un dataframe en un array de numpy.

De Numpy:

`np.hstack((A,B))` fusiona dos matrices A y B añadiendo B al final de A. Deben tener un tamaño compatible.

`np.zeros_like(np_array)` genera un array del mismo tamaño que `np_array` de ceros.

Teneis además la documentación descargada de MLPClassifier y del KNeighborsClassifier que podeis consultar.